

1. Executive System Overview

AeroGlide is a full-featured web-based domestic airline booking and travel management system tailored for top tourist destinations across the Philippines (including Boracay, Coron, Siargao, Mayon Volcano Albay, Chocolate Hills Bohol, Taal Volcano, Malapascua, and Bukidnon). The application provides end-to-end user travel customization (flights, hotel add-ons, car rentals), dynamic fare calculation, promo coupon validation, a live Flight Tracker with boarding pass ticket generation, and a secure Admin Control Panel.

Component	Technology / Specification
Frontend Presentation	HTML5, CSS3 (Vanilla Design System with Glassmorphism & Custom Palettes), JavaScript ES6
Backend Engine	PHP 8.5 (Modular MVC-inspired Architecture, Session Security, Strict Guards)
Database Management	MySQL / MariaDB via PDO (<code>`ag_db()`</code>), Prepared Statements, Relational Foreign Keys
Local Server Environment	WAMP Stack (<code>`localhost/project`</code> and <code>`localhost/website`</code>)
Security Model	Bcrypt Password Hashing (<code>`PASSWORD_BCRYPT`</code>), Role Guards (<code>`admin`/`user`</code>), Session Isolation

2. System Architecture & Folder Layout

The project follows a modular, organized file hierarchy that separates core database models, authentication helpers, reusable UI components (``includes/``), user features (``User/``), and administrative controls (``admin/``).

Folder / File	Function & Responsibility
includes/head.php	Shared HTML head metadata, Google Fonts (Montserrat, Plus Jakarta Sans), and master stylesheet.
includes/navbar.php	Shared top navigation header bar; dynamically renders user greeting, active links, and Admin Panel badge.
includes/footer.php	Shared bottom footer section; contains destination links, user manual anchors, and legal notices.
database/config.php	Central PDO database connection singleton (<code>`ag_db()`</code>) connecting to MySQL <code>`aeroglide`</code> database.
database/function.php	Core utility library: <code>`ag_base_url()`</code> , recursive <code>`asset_find()`</code> , currency formatter, date formatter, and <code>`admin_log()`</code> .

database/validation.php	Server-side form sanitization and coupon eligibility validation (<code>`val_coupon_eligibility`</code>).
auth_helper.php	Authentication & business logic handler: user lookup, login/register, booking retrieval/insertion, coupon tracking.
admin/guard.php	Access control guard requiring authenticated administrator session (<code>`\$user['role'] === 'admin'`</code>).
admin/sidebar.php	Unified admin sidebar navigation bar with single-tab redirect back to the public homepage.
admin/index.php	Admin Dashboard overview displaying total revenue, booking counts, registered users, and recent actions.
admin/bookings.php	Master booking management table with search, status filters (Confirmed/Cancelled), user filter, and inspector.
index.php	Public homepage featuring photo hero slideshow, interactive flight search card, deal packages, and flight tracker.
destination.php	Dynamic package visualizer and price calculator; handles hotel room/car rental add-ons and slideshow galleries.
checkout.php	Step 1 (trip summary & promo code application) and Step 2 (payment method selection & booking submission).
User/dashboard.php	User Dashboard & My Trips area; renders active bookings and generates printable boarding pass tickets.

3. Database Schema & Data Dictionary

The system operates on a relational MySQL database named **aeroglide** containing 4 core tables:

A. users Table (Account Management)

Column Name	Data Type	Key / Constraint	Description
id	INT	PRIMARY KEY, AUTO_INC	Unique user account identifier
username	VARCHAR(50)	UNIQUE, NOT NULL	Unique login handle (lowercase)
name	VARCHAR(100)	NOT NULL	Full name of user / traveler
email	VARCHAR(100)	UNIQUE, NOT NULL	Validated user email address
password	VARCHAR(255)	NOT NULL	Bcrypt password hash (`password_hash`)
role	ENUM('user','admin')	DEFAULT 'user'	Access privilege level
used_coupon	VARCHAR(50)	NULLABLE	Claimed discount code (e.g. 5PEOPLE2026)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account registration timestamp
last_login	DATETIME	NULLABLE	Most recent login timestamp

B. bookings Table (Flight & Package Orders)

Column Name	Data Type	Description
id / booking_ref	INT / VARCHAR(20)	Primary Key & unique reference code (e.g. AG-8F3A21)
user_id	INT (Foreign Key)	Links booking to owner in `users.id`
traveler_name / email / phone	VARCHAR	Contact details of lead traveler
city / mode / adults / cabin	VARCHAR / INT	Destination city, mode (flights/hotels), guest count, cabin class
fare_name / flight_schedule	VARCHAR	Selected fare tier (Smart Saver/Flexi) & flight time schedule
hotel_name / price / nights	VARCHAR / DECIMAL / INT	Selected hotel resort add-on details
car_name / price / days	VARCHAR / DECIMAL / INT	Selected tourist vehicle rental add-on details
subtotal / discount_num / grand_total	DECIMAL(10,2)	Calculated financial summary & applied coupon discount amount
promo_code / payment_method / status	VARCHAR	Applied promo code, payment mode (card/gcash/bank), status (confirmed/cancelled)

4. Key Features & Business Logic Implementation

1. Multi-Tier Destination Package & Pricing Calculator:

Calculates dynamic flight + hotel + car rental totals using cabin tier multipliers (Economy 1.0x, Premium Economy 1.3x, Business 1.8x, First Class 2.5x) and guest counts.

2. Promo Coupon & Discount Validation Engine:

The system enforces server-side coupon validation rules in database/validation.php:

- 5PEOPLE2026: 30% OFF flat discount (unlocked when guest count ≥ 5).
- SEPTDEAL2026: ₱8369;1,500 flat discount off grand total.
- BERFLY1000: ₱8369;1,000 flat discount off grand total.

Upon checkout, claimed coupons are permanently recorded in the user's database record to prevent double-claiming.

3. Real-Time Flight Tracker & Realistic Boarding Pass Ticket:

Allows users to look up flight statuses by Flight Number (e.g. AG-204) or Route. User Dashboard generates high-contrast printable Boarding Passes complete with deterministic SVG barcodes, seat numbers, gate information, and passenger details.

4. Administrator Panel & Management Tools (`admin/`):

Provides administrators with complete oversight over system operations: real-time booking table, filtering by status or specific user ID, detailed booking inspection modals, user role management, revenue analytics reports, and system activity logs.

5. Professor Project Defense Q&A; Cheat Sheet

Use these exact technical answers when your professor asks questions about your website implementation:

Q1: How does your web application connect to the database and manage queries?

Answer: We use PHP Data Objects (PDO) wrapped in a singleton helper function ``ag_db()`` inside ``database/config.php``. All database queries utilize prepared statements (``$stmt->prepare()`` and ``$stmt->execute()``) to prevent SQL injection vulnerabilities.

Q2: How are user passwords secured in the database?

Answer: Passwords are never stored in plain text. When a user registers (``auth_register()``), their password is encrypted using PHP's native ``password_hash($password, PASSWORD_BCRYPT)``. During login, we verify credentials using ``password_verify($password, $user['password'])``.

Q3: How is access control handled between regular users and administrators?

Answer: Access control is enforced at the top of every admin page via ``require_once 'admin/guard.php'``. The guard script checks if ``auth_is_logged_in()`` and ``auth_is_admin()`` return true (verifying ``$_SESSION['user_role'] === 'admin'``). Non-admin users are automatically redirected to the login page.

Q4: How do you prevent code duplication across multiple web pages?

Answer: We modularized shared layout elements into an ``includes/`` directory containing ``head.php``, ``navbar.php``, and ``footer.php``. Global functions (like URL builders, asset lookup, currency formatters) are centralized in ``database/function.php`` and ``auth_helper.php`` guarded with ``function_exists()`` checks.

Q5: How does the system handle image assets when file names vary?

Answer: We built a custom recursive asset finder function ``asset_find(array $terms)`` in ``database/function.php``. It scans the ``/asset`` folder, normalizes filenames, matches keywords, and returns an absolute ``ag_base_url()`` path with ``rawurlencode()`` formatting.

Q6: How does the flight booking calculation work?

Answer: The calculation multiplies the base destination price by the cabin class multiplier (Economy 1.0x to First Class 2.5x), multiplies by the guest count, adds optional hotel/car days, subtracts validated promo coupon discounts, and stores both subtotal and grand total in the ``bookings`` MySQL table.

Q7: What happens when a user forgets their password?

Answer: The user inputs their email/username in ``forgot_password.php``. The system generates a cryptographically secure 64-character token (``bin2hex(random_bytes(32))``), saves it to the ``password_tokens`` table with a 2-hour expiration timestamp, and lets the user safely set a new password via ``reset_password.php``.

Q8: How does the admin panel track changes made by administrators?

Answer: We implemented an audit logging function ``admin_log($action, $target)`` in ``database/function.php``. Whenever an admin modifies a booking status or updates user accounts, an event record is saved to the ``admin_logs`` database table with the admin ID and timestamp.

Q9: What is the relationship between the ``users`` and ``bookings`` tables?

Answer: It is a One-to-Many (1:N) relationship. One registered user can have multiple flight bookings. The ``bookings.user_id`` column acts as a foreign key linking each booking record to ``users.id``.

Q10: Is the web application responsive across different device screen sizes?

Answer: Yes, the application utilizes modern CSS flexbox, CSS grid layouts, relative rem/vh units, and ``@media`` queries in ``style.css`` to ensure full responsiveness across desktop, tablet, and mobile browsers.